RAJALAXMI CHILDREN FOUNDATION · PRATIBHA POSHAK · IMAS PORTAL

Backend Migration to Java & Spring Boot

Architecture & Migration Design Specification • 2026-07-04 • Senior Architect Review

IMAS Backend Migration to Java / Spring Boot — Design

Date: 2026-07-04 **System:** IMAS (PP-Portal) — Management Information System of Rajalaxmi Children Foundation (RCF), Pratibha Poshak program **Scope:** Rewrite the Node.js/Express backend as a Spring Boot application. The React frontend and the PostgreSQL schema (`pp` , ~50 tables) remain unchanged.

---

1. Context and Goals

IMAS automates the Pratibha Poshak talent-search program end to end: NMMS applicant intake and fuzzy merge, shortlisting, exam logistics with PDF hall tickets, evaluation, interviews and home visits, final selection, then the academic lifecycle — cohorts, batches, timetables, attendance, teacher hours, events (Sammelans), device inventory — across five roles: `ADMIN` , `BATCH COORDINATOR` , `TEACHER` , `STUDENT` , `INTERVIEWER` .

The current backend is Express with ~290 active endpoints in 31 route files, raw parameterized SQL via `pg` , and significant accumulated debt:

- ~80% of endpoints have **no authentication**; JWT roles are **never enforced** server-side.
- `/logs` (server logs) and `/Data` (entire file-storage tree) are served publicly.
- Broken, unmounted global error handler; duplicated body parsers; four multer configs; six CSV libraries; three date libraries.
- Verbatim endpoint duplication (evaluation copies custom-list; duplicate student controllers; three generations of commented-out coordinator routes).
- Timetable generation shells out to a Python solver at a hardcoded absolute path.
- No tests, no migration tooling.

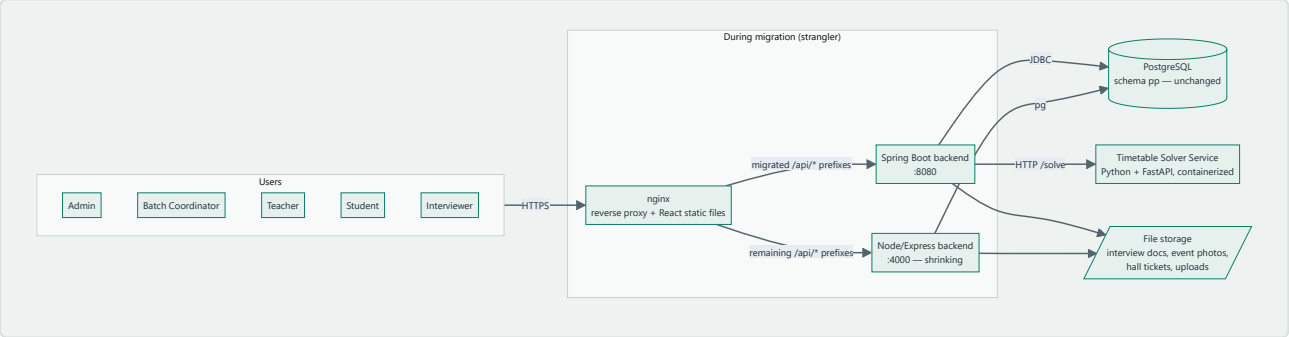
Goals of the migration: maintainability for a small team, a foundation for the v3.0 roadmap (alumni, notifications, staff role), proper security enforcement, and behavioral parity with the frontend's existing API contract.

2. Decisions

Decision	Choice
Migration strategy	**Strangler** — nginx routes migrated `/api/*` prefixes to Spring Boot, the rest to Node, module by module
Data access	**JPA/Hibernate for entities and CRUD; Spring `JdbcClient` native SQL for complex reads** (reports, dashboards, shortlisting, search), ported nearly verbatim

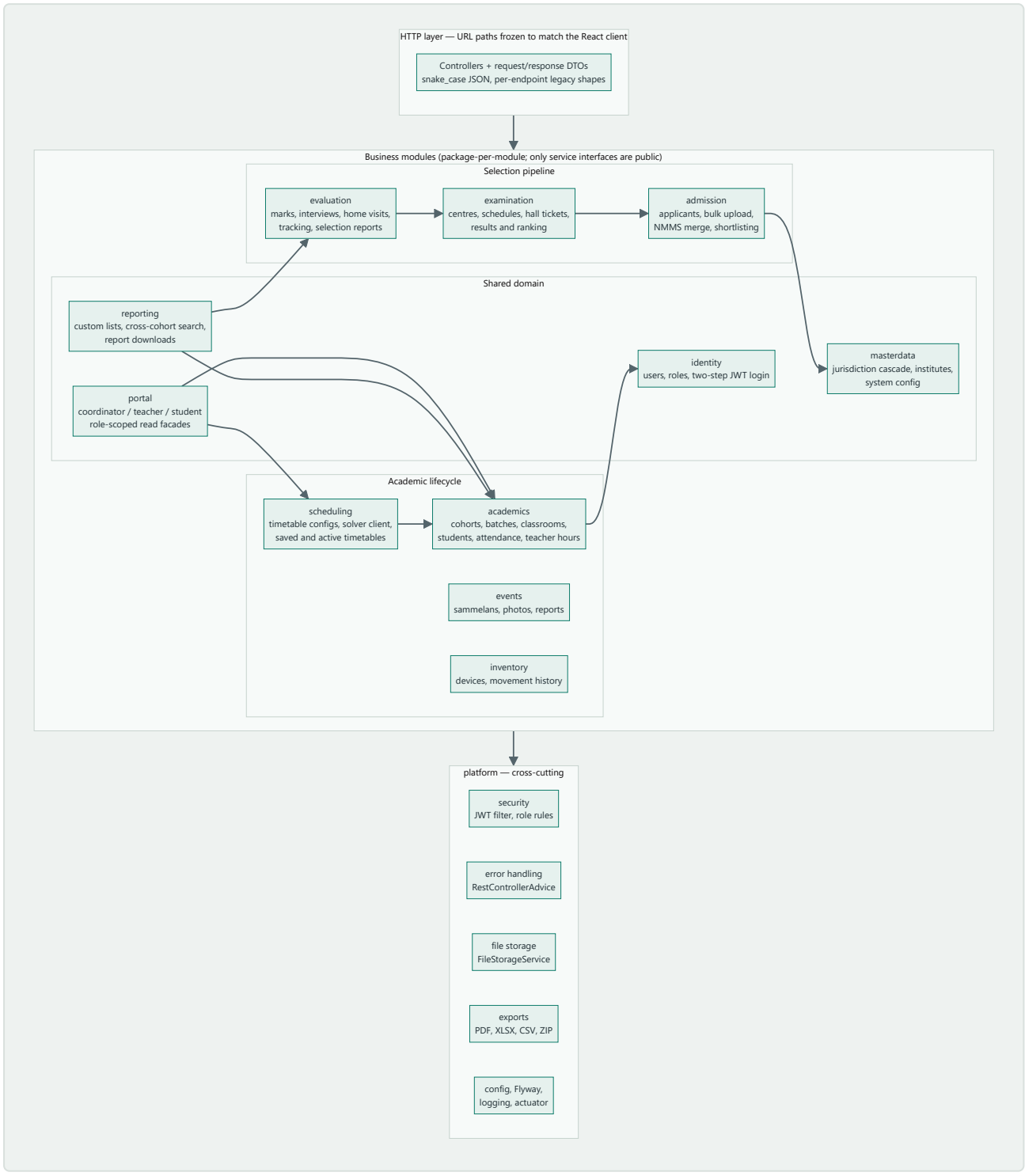
Decision	Choice
Security	**Full enforcement now** — JWT required on all endpoints except public ones; per-module role authorization
Timetable solver	**Keep Python** — wrap `tt.py` in a small containerized HTTP service; Spring calls it
Stack	**Java 21 LTS, Spring Boot 3.x, Maven, single deployable**
Code organization	**Modular monolith** — package per business module, shared `platform` layer, service-interface-only boundaries
Frontend / DB	**Unchanged** — URL paths, JSON shapes, and schema are frozen contracts

3. Target Architecture — System Context



End state: the `NODE` box disappears; nginx routes all of `/api/*` to Spring Boot. JWTs are signed with the same secret and claims layout in both backends, so a user's session survives any cutover.

4. Target Architecture — Modules



Package layout

```
com.rcf.imas
├── platform
│   ├── security        # JWT filter chain, token service, role rules
│   ├── error           # @RestControllerAdvice + legacy-shape exceptions
```

```
│   ├── storage         # FileStorageService over env-driven roots
│   ├── exports         # PDF (OpenPDF), XLSX (Apache POI), CSV (Jackson-CSV), ZIP
│   └── config          # profiles, Jackson snake_case, Flyway, actuator, logging
└── modules
    ├── identity        ├── masterdata      ├── admission
    ├── examination     ├── evaluation      ├── academics
    ├── scheduling      ├── events          ├── inventory
    ├── reporting       └── portal
```

Each module contains `web/` (controllers, DTOs), `service/`, `persistence/` (JPA entities, Spring Data repositories, `JdbcClient` query classes). **Boundary rule:** controllers and persistence are package-private; other modules may call only a module's public service interfaces. This is the rule that prevents re-tangling; optionally enforced with Spring Modulith verification tests.

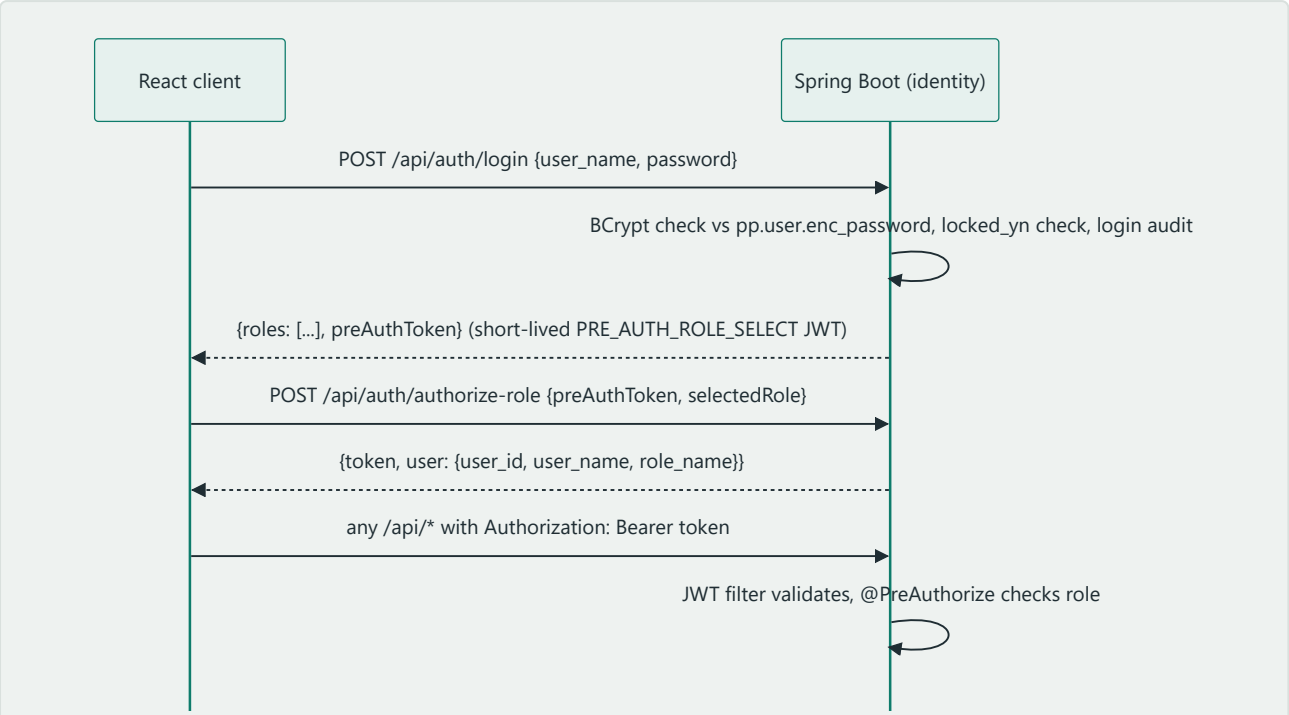
Route ownership (URL prefixes are frozen; assignment is internal)

Module	Owned URL prefixes
identity	`/api/auth`, `/api/users*`, `/api/roles*` (userRole routes)
masterdata	`/api/states`, `/api/divisions-by-state`, `/api/districts-by-division`, `/api/blocks-by-district`, `/api/clusters-by-block`, `/api/institutes-by-cluster`, `/api/districts`, `/api/institutes`, `/api/juris-names`, `/api/system-config`
admission	`/api/applicants`, `/api/bulk-upload`, `/api/merge`, `/api/shortlist/generate`, `/api/shortlist-info`
examination	`/api/exams` (incl. public `/api/exams/hallticket/*`), `/api/results`
evaluation	`/api/evaluation`, `/api/evaluation-dashboard`, `/api/interview`, `/api/tracking`, `/api/selection-reports`
academics	`/api/batches`, `/api/classrooms`, `/api/attendance*`
scheduling	`/api/activetimetable`, timetable generation routes (currently unmounted in Node)
events	event routes currently on bare `/api` (`/api/events*`, event types, photos, reports)
inventory	tab-inventory routes currently on bare `/api`
reporting	`/api/custom-list`, search routes, `/api/search*`
portal	`/api/coordinator`, `/api/teacher`, `/api/student`

Notes: - The six Node routers mounted at bare `/api` become explicit full-path `@RequestMapping`s in Spring — no path changes, no ordering ambiguity. - The evaluation module's nine endpoints duplicated verbatim from custom-list keep their URLs but delegate to the shared `reporting` service. Duplicate controllers (`studentController1.js` fork, `reports.js` vs coordinator copies) are merged into single services. - `portal` holds the role-scoped read aggregations (coordinator 38, teacher 9, student 10 endpoints) so core modules don't accumulate per-role query variants.

5. Security Design

Stateless Spring Security filter chain:



- **Public endpoints:** `POST /api/auth/login`, `POST /api/auth/authorize-role`, `GET /api/exams/hallticket/{no}`. Everything else requires a valid Bearer JWT.
- **Token compatibility:** same signing secret, `role_name` claim, and numeric `exp` as Node, so tokens are interchangeable across both backends during migration and remain decodable by the client's `jwt-decode`.
- **Role authorization matrix** (enforced via `@PreAuthorize`; role strings exactly as stored):

Module	ADMIN	BATCH COORDINATOR	TEACHER	STUDENT	INTERVIEWER
identity (user/role admin)	✓	–	–	–	–
masterdata reads	✓	✓	✓	✓	✓
masterdata writes, system-config	✓	–	–	–	–
admission, examination, evaluation admin	✓	–	–	–	–
interview feedback endpoints	✓	–	–	–	✓
academics management	✓	✓	–	–	–
scheduling management	✓	✓	–	–	–
events, inventory, reporting	✓	✓ (read subset)	–	–	–
portal/coordinator	✓	✓	–	–	–
portal/teacher	✓	–	✓	–	–

Module	ADMIN	BATCH COORDINATOR	TEACHER	STUDENT	INTERVIEWER
portal/student	✓	–	–	✓	–

The exact endpoint-level matrix is finalized per phase during implementation planning (some coordinator/teacher reads overlap); the default is deny.

- **Static-content lockdown:** `/logs` is no longer served. `/Data` is replaced by authorized, scoped download endpoints through `FileStorageService`. Profile photos, event photos, and hall tickets are served through authenticated endpoints (or nginx `internal` redirects), preserving the URL paths the frontend uses.
- **Preserved behaviors:** anti-enumeration login responses, account locking, login audit records.
- **Risk control:** before each phase's cutover, audit that phase's frontend pages for raw `fetch()` calls missing the Authorization header (the global axios interceptor covers axios calls).

6. API Compatibility Contract (invariants)

The React client is unchanged, therefore:

1. All paths under `/api/`, exact spellings (kebab-case paths, snake_case route params like `:nmms_reg_number`).
2. **snake_case JSON everywhere** — app-wide Jackson `SnakeCaseStrategy`.
3. **Per-endpoint response shapes reproduced, not normalized**: bare arrays vs `{data: [...]}` vs `{success, data}`; bulk upload returns `{totalRecords, insertedRecords, validationErrors, dbErrors, status, logFile}`.
4. Error bodies preserved per endpoint where the client reads `error` / `message` / `msg` / `logs` / `logFile`; login failure body includes `error`.
5. Exact role strings: `ADMIN`, `BATCH COORDINATOR`, `TEACHER`, `STUDENT`, `INTERVIEWER`.
6. Dates as ISO `YYYY-MM-DD` strings.
7. Blob endpoints (hall tickets, report/CSV/XLSX/PDF downloads) return raw binary with correct `Content-Type`, never JSON wrappers; multipart field names unchanged (`file`).
8. Status-code semantics preserved, including `409` for timetable/exam conflict flows and `201` where currently returned.
9. Literal status strings preserved (e.g. interview `"Assigned"`, `"Reassigned"`, `"RESCHEDULED"`, `"Scheduled"`).
10. `GET /api/system-config/active` returns an array of configs each containing `academic_year`.

7. Platform Layer Details

- **Error handling:** one `@RestControllerAdvice`; controllers throw typed exceptions carrying the legacy body shape where the client depends on it; a consistent `{message}` default otherwise.
- **File storage:** `FileStorageService` over env roots (`FILE_STORAGE_PATH`, `EVENT_STORAGE_PATH`, `PROFILE_PHOTOS_ROOT`, `USER_PROFILE_ROOT`); on-disk layout preserved (`Interview-data/`, `Home-verification-data/cohort-{year}/`, `uploads/events/{photos,reports}`, `public/halltickets`) because

DB rows store those paths. One upload policy (size limits, content-type whitelist) replaces the four multer configs.

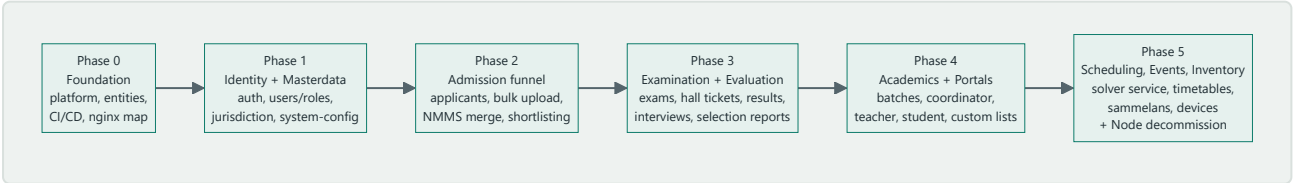
- **Exports:** OpenPDF (open-source rule) for hall tickets, selection reports, custom lists, timetables; Apache POI for XLSX; Jackson-CSV for CSV; `java.util.zip` for bulk hall-ticket archives. The abandoned JasperStarter vendor folder is not carried over.
- **Data access:** JPA entities for all ~50 tables (generated from the live schema, then hand-tuned); Spring Data repositories for CRUD; `JdbcClient` classes for the complex report/dashboard/shortlist/search SQL ported from Node models. Transaction boundaries from the 18 multi-step Node write flows become `@Transactional` service methods. The dynamic `DELETE FROM ${table}` in the merge flow becomes a whitelisted enum of staging tables.
- **Migrations:** Flyway, baselined on the existing schema (V1 = no-op baseline). No schema changes in this project.
- **Config/ops:** Spring profiles `dev` / `prod`; env-var names kept where practical so Docker/compose changes are minimal; Actuator health for nginx; structured request logging filter (user, role, IP, path) replacing the custom middleware — logs written to files, never served over HTTP.
- **Validation:** Bean Validation on request DTOs, mirroring (not tightening) current accepted inputs to avoid breaking existing client payloads; genuinely missing critical validation may be added where it cannot break the UI.

8. Timetable Solver Service

The existing, validated `tt.py` constraint solver is wrapped in a minimal FastAPI service:

- `POST /solve` — input: teacher availability/skills, subjects, batches, slot config (same data the Node service marshalled); output: feasible timetable or infeasibility reasons. Long runs handled with a job id + `GET /solve/{id}` polling.
- Containerized alongside the backend in docker-compose; no hardcoded interpreter paths.
- The Spring `scheduling` module calls it via `RestClient`; solver inputs/outputs persist in the existing `time_table_config_file` / `time_table_solution` tables.
- The solver remains independently replaceable (e.g. by Timefold) later without touching module boundaries.

9. Migration Plan (strangler phases)



Phase	Prefixes cut over	Why this order
0	none	Skeleton, platform, JPA entities for all tables, CI building both backends, nginx routing map
1	`/api/auth`, users/roles, `/api/system-config`, jurisdiction cascade, `/api/districts`, `/api/institutes`, `/api/juris-names`	Small and foundational; exercises the full platform stack and the riskiest compatibility surface (two-step login) while per-endpoint stakes are low
2	`/api/applicants`, `/api/bulk-upload`, `/api/merge`, `/api/shortlist/generate`, `/api/shortlist-info`	Validates transactions, CSV parsing, staging-table merge, fuzzy matching (Apache Commons Text)
3	`/api/exams`, `/api/results`, `/api/evaluation*`, `/api/interview`, `/api/tracking`, `/api/selection-reports`	The PDF/export-heavy phase, incl. public hall-ticket endpoint
4	`/api/batches`, `/api/classrooms`, `/api/coordinator`, `/api/teacher`, `/api/student`, search, `/api/custom-list`	Largest endpoint count; patterns well-worn by now
5	`/api/activetimetable`, timetable generation (new solver service), events, tab inventory	Then Node serves zero prefixes for two weeks → removed from compose; dead code archived

Each phase's exit criteria: parity snapshots pass, that module's frontend pages smoke-tested against Spring Boot, `fetch()` header audit done, rollback = one nginx map edit.

10. Testing and Parity Strategy

The Node backend has no tests; the contract is captured empirically per phase:

1. **Record:** run the module's key frontend flows against Node; capture request/response pairs (nginx mirror or capture script).
2. **Replay:** run the same requests against Spring Boot; assert shape compatibility — field names, wrapper style, status codes — ignoring volatile values (timestamps, generated ids).
3. **New tests:** `@WebMvcTest` for controller/security rules (every endpoint's auth requirement gets a test), Testcontainers-PostgreSQL integration tests for transactional and bulk flows, unit tests for merge fuzzy-matching and shortlist computation against known Node outputs.

The Java codebase therefore starts life with the regression suite the Node one never had.

11. Risks and Mitigations

Risk	Mitigation
Enforcing auth breaks a frontend call that never sent a token	Per-phase `fetch()` audit; strangler rollback is one nginx edit
Subtle response-shape drift (wrapper style, field casing)	Recorded-snapshot replay per endpoint before cutover; snake_case set globally
Complex SQL behaves differently under JPA	Complex reads stay native SQL via `JdbcClient`, ported verbatim; JPA only for CRUD

Risk	Mitigation
Two backends in production simultaneously	Shared JWT secret/claims; shared DB and file storage; phase scope kept module-coherent so cross-backend calls are never needed
Solver service integration surprises	Timetable routes are currently unmounted in Node — no live users; phase 5 timing gives slack
PDF byte-for-byte differences (hall tickets, reports)	Visual/manual review per template during its phase; exact byte parity is not a goal, layout fidelity is

12. Out of Scope

- Any PostgreSQL schema change (Flyway baseline only).
- Any React frontend change (except none-required; the `fetch()` audit may *recommend* fixes but the design does not depend on them).
- v3.0 roadmap features (alumni, notifications, staff role, AI) — the module structure is designed so they land as new modules later.
- Mobile app (React Native) — consumes the same frozen API.
- Rewriting the solver in Java (Timefold) — deliberately deferred; boundary preserved.